OPEN LEARNING ROADMAP · NOVEMBER 2026

From DevOps engineer to AI infrastructure engineer

A 26-week plan for DevOps, cloud and platform engineers who want to move into AI infrastructure. You keep the skills that already pay, add the layer companies are short of, which is running AI models in production, and build public proof that you can do it.

HORIZON	EFFORT	BUDGET	TARGET ROLE
26 weeks	8–10 h / week	$25–60 / month	AI Platform Eng.

01 · START HERE

Who this is for

This roadmap is for engineers who already run infrastructure and want to run AI workloads too. It assumes the starting point below. If you're ahead on a row, skip or shorten the phase that covers it.

AREA	ASSUMED STARTING POINT	APPROACH IN THIS PLAN
Experience	1–5 years in DevOps, SRE or cloud engineering	Build on what you already do well rather than starting over
Cloud	Comfortable with one cloud (examples use AWS)	Stay on the cloud you know. The ideas transfer to Azure (AKS) and GCP (GKE).
Kubernetes	You deploy apps but don't operate clusters	Learn to operate clusters and pass the CKA. GPU scheduling depends on this.
Python	Little or none	Four focused weeks up to writing FastAPI services and async scripts
ML knowledge	None needed	Learn the infrastructure side of models: memory, serving, cost. No heavy maths.
Hardware	An ordinary laptop, no GPU	Rent cloud GPUs by the hour. Keep local work to kind clusters and API-based tools.

AREA	ASSUMED STARTING POINT	APPROACH IN THIS PLAN
Time	8–10 hours a week alongside a full-time job	Six phases over 26 weeks, each ending in something you can show

THE CORE IDEA

Most DevOps engineers learning AI collect course certificates. You will collect **numbers**: cost per million tokens, time to first token, GPU utilization, and money saved for your company. Hiring managers remember numbers, and they're difficult to fake.

02 · START HERE

The AI job market for DevOps engineers

AI did not replace DevOps. It added an expensive new kind of workload. A GPU node costs 10–50× more per hour than a typical web node, model files are gigabytes in size, and traffic is measured in tokens rather than requests. Companies need people who can run these workloads reliably and cheaply, and very few people have both the infrastructure skills and the AI knowledge.

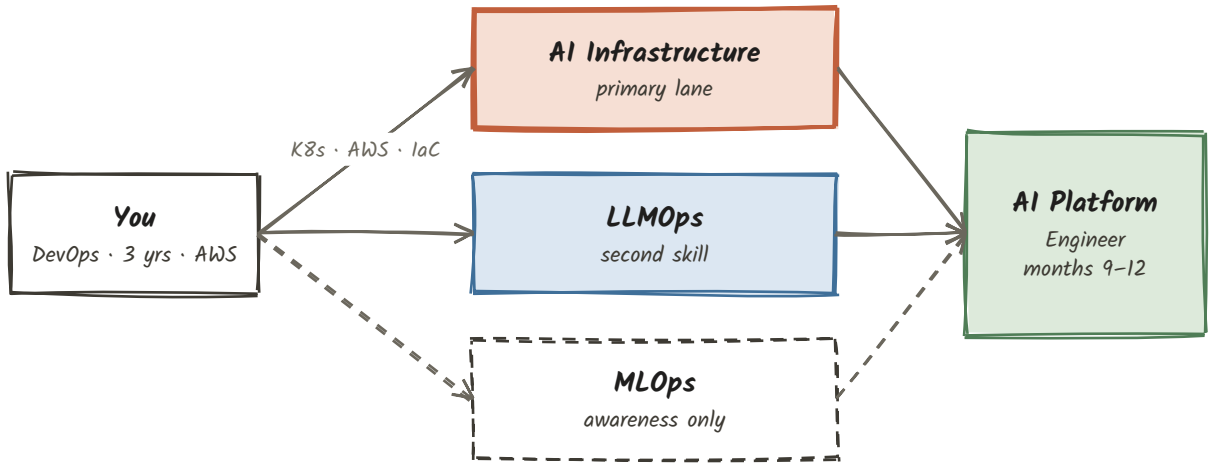


Fig. 1 Three lanes open to a DevOps engineer. You take the AI infrastructure lane, add LLMOps as a second skill, and learn only enough MLOps to talk with data scientists.

PRIMARY · AI INFRASTRUCTURE

Run the machines that run models

GPU clusters, model serving with vLLM and KServe, autoscaling, and GPU cost control. It builds on your Kubernetes, Terraform and AWS skills, and it's the hardest of the three to automate away.

SECOND SKILL · LLMOPS

Run the apps built on models

Gateways, RAG pipelines, evaluations, tracing, guardrails and token budgets. Every company that calls LLM APIs needs this now.

AWARENESS · MLOPS

Pipelines for training models

MLflow, Kubeflow and feature stores. Learn the vocabulary, but don't specialize. Classic MLOps tooling is consolidating into managed platforms.

03 · START HERE

The AI infrastructure stack

This is the map for the whole plan. Today you know the orchestration layer from the application side. By week 26 you'll have run every layer yourself and measured it.

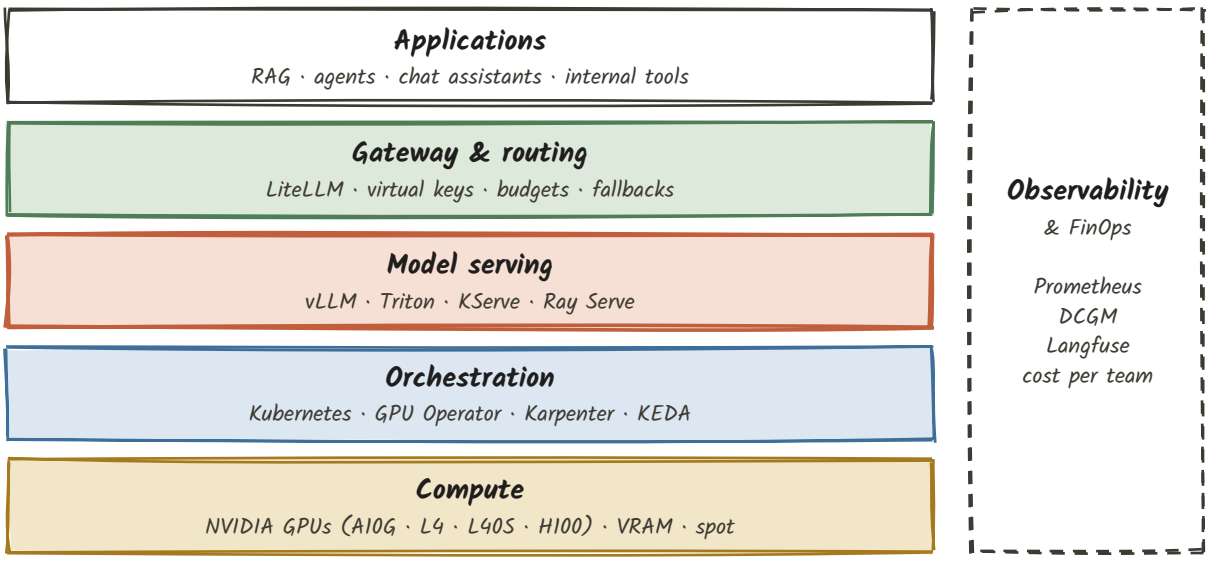


Fig. 2 Each layer maps to a phase: compute and orchestration in P2 and P4, serving in P4, gateway in P3, applications in P5, observability and FinOps in P3–P6.

GPU memory, the one formula to learn first

Most AI infrastructure decisions start with a single question: does the model fit in GPU memory? Learn this before you touch a GPU.

```
VRAM ≈ parameters × bytes per parameter × 1.2  (+ KV cache for long
contexts and many users)
```

MODEL SIZE	FP16 (2 B)	INT8 (1 B)	INT4 / AWQ (0.5 B)	SMALLEST PRACTICAL AWS GPU
1.5B	~4 GB	~2 GB	~1 GB	Any GPU, even g4dn (T4 16 GB)
7–8B	~18 GB	~9 GB	~5 GB	g5 (A10G 24 GB) or g6 (L4 24 GB)
14B	~34 GB	~17 GB	~9 GB	g6e (L40S 48 GB), or g5 with AWQ
70B	~168 GB	~84 GB	~42 GB	p4d / p5 (multiple A100 / H100 GPUs)

Weights plus about 20% overhead. Leave additional headroom for the KV cache, which grows with context length × concurrent requests.

04 · START HERE

Skill gap map

You don't need a data science degree. You need the eight skills below, and you already have most of what each one is built on.

SKILL	TODAY	NEEDED FOR THE ROLE	CLOSED IN
Python	Little or none	Write APIs, async load tests, small operators and glue code	P1
LLM basics	Uses ChatGPT	Tokens, context, KV cache, quantization, VRAM math, inference vs. training	P1
K8s operations	Deploys apps	Scheduling, taints, node pools, storage, troubleshooting, CKA level	P2
LLM gateway	None	Central keys, budgets, fallbacks, rate limits, tracing	P3
GPU on K8s	None	GPU Operator, device plugin, Karpenter GPU pools, DCGM metrics	P4
Model serving	None	vLLM, batching, autoscaling on queue depth, cold starts, benchmarking	P4

SKILL	TODAY	NEEDED FOR THE ROLE	CLOSED IN
RAG and evals	None	Embeddings, vector DB, eval sets in CI, guardrails	P5
AI FinOps	General AWS cost awareness	Cost per 1M tokens, GPU utilization, spot strategy, showback per team	P3–P6

05 · ROADMAP

The 26-week timeline

There are six phases. Each one ends with something you can show: a script, a certificate, a working system at your company, or a public write-up with numbers.

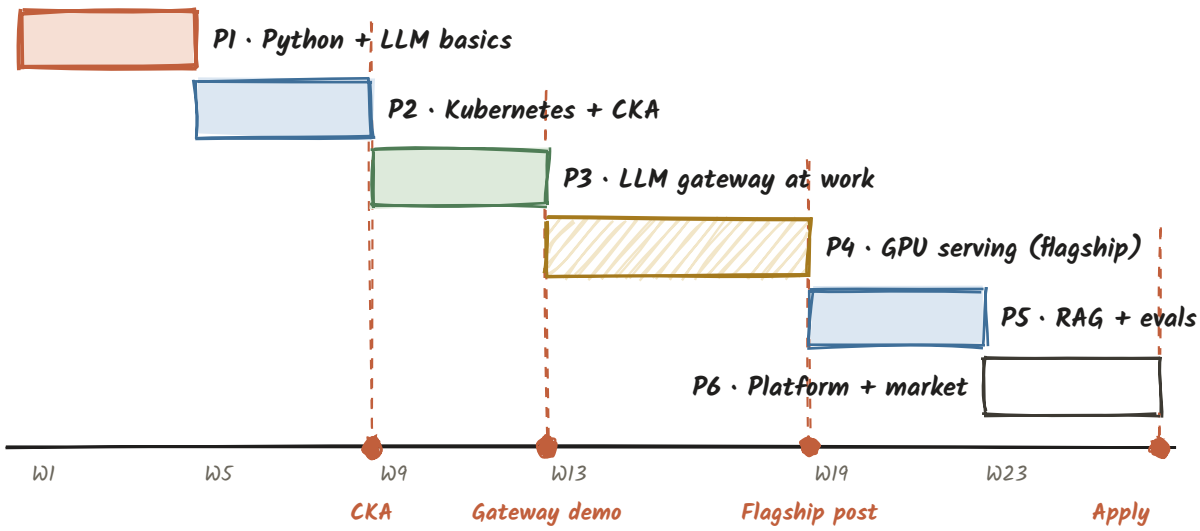


Fig. 3 Phase 4 gets the most time because it produces your flagship portfolio project. The markers on the axis are your four milestones.

PHASE 1 · WEEKS 1–4

Python and LLM basics

Python | FastAPI | LLM concepts | Cost: ~$4 in API tokens

Python is the language of the AI ecosystem. vLLM, LiteLLM, Ray, Kubeflow and every evaluation tool are written in it or configured with it. Learn it through ops problems you already understand, not through generic exercises.

WEEK	LEARN	BUILD	OUTPUT
W1	Types, functions, dicts and lists, files, `venv`, `pip`, `subprocess`, JSON	Pod restart reporter (below)	First repo: `ops-python`
W2	`argparse`, `httpx`, `boto3`, exceptions, logging, `pytest`	CLI that lists EC2 spend by tag using Cost Explorer	Tests passing in GitHub Actions
W3	FastAPI, Pydantic, Docker multi-stage builds, the OpenAI-compatible API format	Log summarizer API that calls an LLM, deployed to kind	Container image plus Helm chart
W4	Tokens, context window, temperature, embeddings, KV cache, quantization, inference vs. training	VRAM calculator script using the formula in section 03	**Post #1:** "How much GPU does a 7B model need?"

PYTHON restarts.py · week 1

```python
import json
import subprocess

out = subprocess.run(
    ["kubectl", "get", "pods", "-A", "-o", "json"],
    capture_output=True, text=True, check=True,
).stdout

for pod in json.loads(out)["items"]:
    for cs in pod["status"].get("containerStatuses", []):
        if cs["restartCount"] > 3:
            ns, name = pod["metadata"]["namespace"], pod["metadata"]["name"]
            print(f"{ns}/{name}  container={cs['name']}  restarts={cs['restartCount']}")
```

PYTHON app.py · week 3 log summarizer

```python
from fastapi import FastAPI
from openai import OpenAI
from pydantic import BaseModel

app = FastAPI()
client = OpenAI()  # in Phase 3, point base_url at your own LLM gateway


class Logs(BaseModel):
    text: str


@app.post("/summarize")
def summarize(logs: Logs):
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "You are an SRE. List the errors and the likely root cause in 5 bullets."},
            {"role": "user", "content": logs.text[-12000:]},  # keep the newest lines; tokens cost money
        ],
    )
    return {"summary": resp.choices[0].message.content, "tokens": resp.usage.total_tokens}
```

PHASE 1 IS DONE WHEN

- [ ] You can write a 50-line Python script without copying from a tutorial.
- [ ] You can explain, without notes, why a 7B model in FP16 does not fit on a 16 GB GPU.
- [ ] Your log summarizer runs in a kind cluster and reports token usage on every call.

PHASE 2 · WEEKS 5–8

Kubernetes operations and the CKA

Cluster operations | Scheduling | CKA exam | Cost: exam fee + ~$6 in VMs

GPU workloads are a scheduling problem first. Taints, tolerations, node affinity, resource limits and priority classes decide which pod gets a GPU that costs $1 an hour. Everything in Phase 4 depends on you knowing these well.

WEEK	LEARN	PRACTICE	WHY IT MATTERS FOR AI
W5	Control plane, kubeadm, etcd backup and restore, upgrades	Build a two-node cluster with kubeadm on spot VMs, then destroy it	Knowing how clusters break makes GPU node debugging possible
W6	Taints and tolerations, affinity, requests and limits, PriorityClass, DaemonSets	Simulate a "GPU" node with a taint and an extended resource	This is exactly how `nvidia.com/gpu` scheduling works
W7	Services, Ingress, NetworkPolicy, PV, PVC, StorageClass, troubleshooting	Killercoda CKA scenarios, debugging broken clusters	Model weights need fast storage, and inference needs ingress
W8	Exam speed: `kubectl` imperative commands, aliases, docs navigation	Two killer.sh mock exams, then take the exam	**Milestone:** CKA certified

LEARN ALONGSIDE THE EXAM

Helm chart authoring, CRDs and the operator pattern (read the code of one small operator), and Karpenter concepts. The CKA doesn't test these, but Phase 4 uses all of them.

PHASE 3 · WEEKS 9–12

An LLM gateway at your company

LiteLLM | Langfuse | FinOps | First internal win

Most companies already call LLM APIs. That usually means API keys copied into many services, nobody knowing which team spends what, and outages whenever a provider goes down. A gateway fixes all three, and building one is mostly DevOps work. It's the fastest way to become known as the AI infrastructure person at your company.

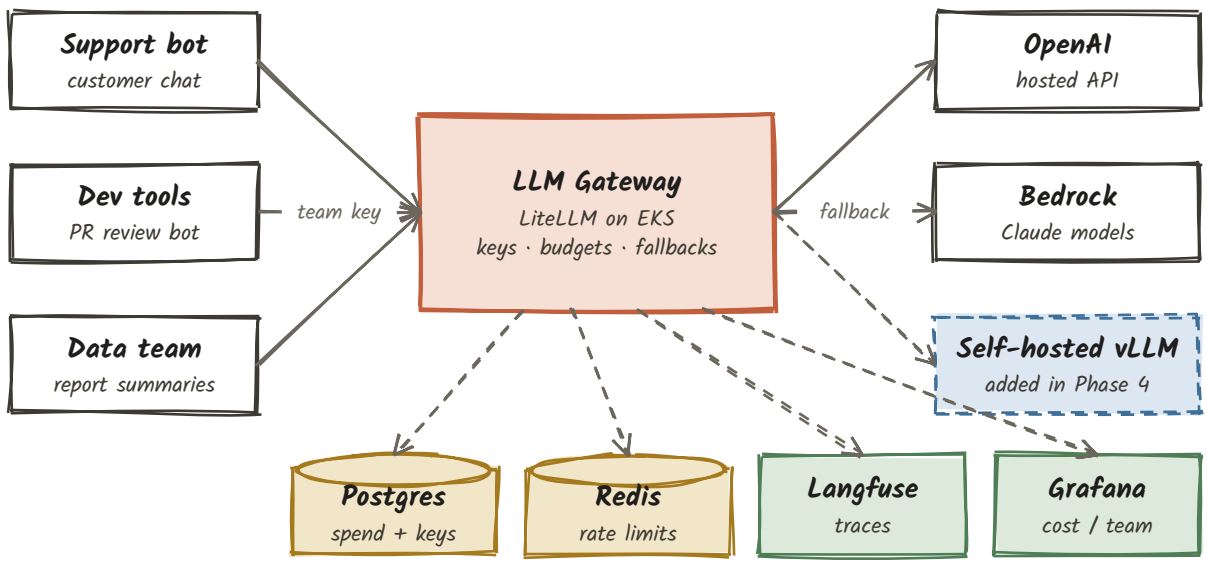


Fig. 4 Each team gets a virtual key with its own budget. The gateway handles retries, fallbacks and cost tracking. In Phase 4 your own vLLM becomes another backend behind it.

WEEK	DO	OUTPUT
W9	Find out which services call which LLM API, how keys are stored, and the monthly spend. Write a one-page proposal (template below).	Proposal sent to your lead
W10	Deploy LiteLLM on EKS with Helm, using RDS Postgres, Redis and secrets from AWS Secrets Manager. Create one virtual key and budget per team.	Gateway running in a dev or staging environment
W11	Add fallbacks, retries and rate limits. Send traces to Langfuse. Build a Grafana dashboard for p95 latency, tokens, errors and cost per team.	Dashboard screenshot

WEEK	DO	OUTPUT
W12	Pilot with one friendly team for a week, then demo the numbers.	**Milestone:** internal demo. **Post #2**

YAML litellm-config.yaml

```yaml
model_list:
  - model_name: chat-default              # what apps ask for
    litellm_params:
      model: openai/gpt-4o-mini
      api_key: os.environ/OPENAI_API_KEY
  - model_name: chat-default              # same name = fallback / load-balance pool
    litellm_params:
      model: bedrock/<model-id-enabled-in-your-account>
      aws_region_name: us-east-1
  - model_name: chat-private              # added in Phase 4
    litellm_params:
      model: hosted_vllm/Qwen/Qwen2.5-7B-Instruct-AWQ
      api_base: http://vllm-qwen.inference.svc:8000/v1

router_settings:
  routing_strategy: simple-shuffle
  num_retries: 2

litellm_settings:
  success_callback: ["langfuse"]
  failure_callback: ["langfuse"]

general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
  database_url: os.environ/DATABASE_URL
```

BASH install and issue a team key

```bash
helm install llm-gateway oci://ghcr.io/berriai/litellm-helm \
  --namespace ai-gateway --create-namespace -f values.yaml

# one key per team: budget in USD, reset every 30 days, 120 requests/min
curl -s https://llm-gateway.internal/key/generate \
  -H "Authorization: Bearer $LITELLM_MASTER_KEY" \
  -H "Content-Type: application/json" \
  -d '{"team_id": "support-bot", "max_budget": 200, "budget_duration": "30d", "rpm_limit":
120}'
```

One-page proposal template

PROBLEM

"We call LLM APIs from N services with M separate keys. We can't see spend per team, keys are hard to rotate, and a provider outage takes features down."

PROPOSAL

"An internal gateway on our existing EKS cluster. One endpoint, a key per team, automatic fallback to a second provider, and a cost dashboard."

COST AND EFFORT

"About 2 vCPU of capacity and a small RDS instance. Three weeks of my time. No changes to application code apart from the base URL."

SUCCESS MEASURE

"After 30 days: 100% of LLM spend visible per team, fewer than 1% failed requests, and one provider outage absorbed with no user impact."

PHASE 4 · WEEKS 13–18 · FLAGSHIP

Serving open models on GPU Kubernetes

vLLM | Karpenter GPU pools | KEDA | Benchmarking | Cost: ~$25–35 in spot GPU hours

This project sets you apart from other DevOps engineers. You'll serve an open-weight model on EKS GPU spot nodes, autoscale it on queue depth, benchmark it honestly, and compare its cost per million tokens with the API your company pays for today.

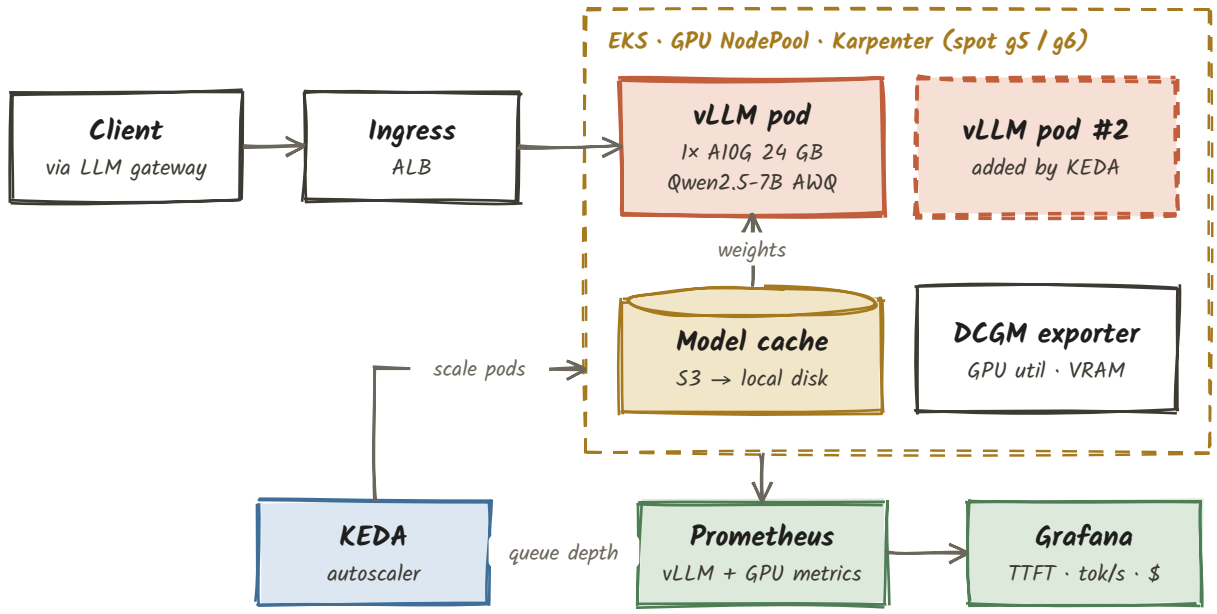


Fig. 5 The flagship architecture. KEDA reads vLLM's waiting-request count from Prometheus and adds pods. Karpenter then adds spot GPU nodes for those pods and removes them when they're idle.

WEEK	DO	OUTPUT
W13	GPU basics: CUDA, drivers, device plugin vs. GPU Operator, MIG and time-slicing. Rent a RunPod GPU for an hour and run vLLM in plain Docker.	First tokens from a GPU you control
W14	Terraform an EKS cluster with Karpenter and a GPU NodePool (g5/g6, spot first). Install the NVIDIA device plugin or GPU Operator and the DCGM exporter.	`terraform apply` in 20 minutes, `destroy` in 10
W15	Deploy vLLM with a 7B AWQ model, health probes and `/dev/shm`. Cache weights to cut cold starts. Register it in your gateway.	OpenAI-compatible endpoint on your cluster

WEEK	DO	OUTPUT
W16	KEDA autoscaling on `vllm:num_requests_waiting`. Measure cold start from pod scheduled to first token.	Scaling graph from 1 to 2 replicas
W17	Benchmark at concurrency 1, 8 and 32: throughput, TTFT p50/p95, and cost per 1M tokens. Compare AWQ with FP16.	Benchmark table and charts
W18	Polish the repo README with the architecture diagram, results and reproduction steps.	**Milestone: Post #3, your flagship**

YAML karpenter-gpu-nodepool.yaml

```yaml
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: gpu
spec:
  template:
    spec:
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: gpu
      requirements:
        - key: karpenter.k8s.aws/instance-family
          operator: In
          values: ["g5", "g6"]          # A10G / L4, 24 GB each
        - key: karpenter.k8s.aws/instance-size
          operator: In
          values: ["xlarge", "2xlarge"]
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]  # spot first; on-demand as fallback
      taints:
        - key: nvidia.com/gpu
          effect: NoSchedule             # only GPU workloads land here
  limits:
    nvidia.com/gpu: 2                    # a hard cap on GPUs is a hard cap on spend
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 5m
```

YAML vllm-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-qwen
  namespace: inference
spec:
  replicas: 1
  selector:
    matchLabels: { app: vllm-qwen }
  template:
    metadata:
      labels: { app: vllm-qwen }
    spec:
      nodeSelector:
        karpenter.sh/nodepool: gpu
      tolerations:
        - key: nvidia.com/gpu
          operator: Exists
          effect: NoSchedule
      containers:
        - name: vllm
          image: vllm/vllm-openai:v0.6.3      # pin the tag you tested
          args:
            - --model=Qwen/Qwen2.5-7B-Instruct-AWQ
            - --max-model-len=8192
            - --gpu-memory-utilization=0.90
          ports:
            - containerPort: 8000
          resources:
            requests: { cpu: "3", memory: 12Gi }
            limits:   { memory: 13Gi, nvidia.com/gpu: 1 }
          readinessProbe:
            httpGet: { path: /health, port: 8000 }
            initialDelaySeconds: 60
            periodSeconds: 10
          volumeMounts:
            - { name: hf-cache, mountPath: /root/.cache/huggingface }
            - { name: shm, mountPath: /dev/shm }
      volumes:
        - name: hf-cache
          emptyDir: {}                        # upgrade: pre-load from S3 in an init
container
        - name: shm
          emptyDir: { medium: Memory, sizeLimit: 2Gi }
```

YAML keda-scaledobject.yaml

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: vllm-qwen
  namespace: inference
spec:
  scaleTargetRef:
    name: vllm-qwen
  minReplicaCount: 1
  maxReplicaCount: 2
  cooldownPeriod: 300
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus-server.monitoring:80
        query: sum(vllm:num_requests_waiting{namespace="inference"})
        threshold: "5"      # more than 5 queued requests per pod: add a pod
```

Benchmark it yourself

This script also serves as Python practice: async code, streaming responses and percentiles. For rigorous numbers later, use vLLM's own benchmark tool or GuideLLM.

PYTHON bench.py

```python
import asyncio, json, statistics, time
import httpx

URL = "http://localhost:8000/v1/chat/completions"   # kubectl port-forward svc/vllm-qwen 8000
MODEL = "Qwen/Qwen2.5-7B-Instruct-AWQ"
PROMPT = "Explain Kubernetes taints and tolerations in 150 words."


async def one_request(client):
    body = {"model": MODEL, "messages": [{"role": "user", "content": PROMPT}],
            "max_tokens": 256, "stream": True}
    start, ttft, tokens = time.perf_counter(), None, 0
    async with client.stream("POST", URL, json=body) as resp:
        async for line in resp.aiter_lines():
            if not line.startswith("data: ") or line.endswith("[DONE]"):
                continue
            chunk = json.loads(line[6:])
            if chunk["choices"] and chunk["choices"][0]["delta"].get("content"):
                ttft = ttft or time.perf_counter() - start
                tokens += 1                     # approx: one streamed chunk = one token
    return ttft, tokens


async def run(concurrency, total=64):
    sem = asyncio.Semaphore(concurrency)
    async with httpx.AsyncClient(timeout=180) as client:
        async def guarded():
            async with sem:
                return await one_request(client)
        t0 = time.perf_counter()
        results = await asyncio.gather(*[guarded() for _ in range(total)])
        wall = time.perf_counter() - t0
    ttfts = sorted(r[0] for r in results if r[0])
    tok_s = sum(r[1] for r in results) / wall
    p95 = ttfts[int(0.95 * (len(ttfts) - 1))]
    print(f"c={concurrency:>3}  {tok_s:7.1f} tok/s  TTFT p50={statistics.median(ttfts):.2f}s p95={p95:.2f}s")


for c in (1, 8, 32):
    asyncio.run(run(c))
```

Turn throughput into money

```
cost per 1M tokens = GPU price per hour ÷ (tokens per second × 3,600) × 1,000,000
```

EXAMPLE (CHECK CURRENT AWS PRICING)	$/HOUR	THROUGHPUT	$ PER 1M TOKENS
g5.xlarge on-demand, light load	~1.01	150 tok/s	~1.87
g5.xlarge on-demand, batched at c=32	~1.01	1,000 tok/s	~0.28
g5.xlarge spot, batched at c=32	~0.40	1,000 tok/s	~0.11

THE INSIGHT TO PUBLISH

Self-hosting is only cheap when the GPU stays busy. At low traffic, a small hosted API model often costs less. Showing *where* the break-even point is, with your own measurements, is the kind of judgment senior engineers are hired for.

PHASE 5 · WEEKS 19–22

RAG, evals and AI security

pgvector or Qdrant | Evals in CI | Guardrails

Now add the LLMOps layer on top of your platform. Build an **ops runbook assistant** that answers on-call questions from your team's own runbooks. Treat it like any production service: ship it with evals, tracing and security controls.

WEEK	DO	OUTPUT
W19	Learn embeddings and chunking. Run pgvector (or Qdrant) on Kubernetes. Write an ingestion CronJob that indexes runbooks from Git.	Index refreshes nightly
W20	Build a FastAPI RAG service that calls models through your gateway. Trace every step in Langfuse.	Answers that cite the runbook they came from
W21	Write a golden set of 30 questions and expected answers. Score them with an LLM-as-judge or Ragas in GitHub Actions, and fail the build when the score drops.	Eval badge in the README
W22	Security: the OWASP Top 10 for LLM applications, prompt injection, PII redaction, NetworkPolicies, and secrets handling.	Threat model page. **Post #4**

PHASE 6 · WEEKS 23–26

Platform polish and the job market

Argo CD | GPU FinOps | Résumé | Interviews

WEEK	DO	OUTPUT
W23	GitOps for models: Argo CD deploys model versions, with a canary rollout of a new model behind the gateway.	Model upgrade with zero downtime
W24	GPU FinOps: utilization from DCGM, idle GPU alerts, spot interruption handling, cost showback per team.	"GPU waste" dashboard
W25	Rewrite your résumé and LinkedIn around outcomes. Pin your 3 repos. Write one case study per project.	Profile ready
W26	Mock system design interviews (see section 10). Apply to 10 carefully chosen roles and ask for referrals.	**Milestone: applications sent**

Résumé bullets you'll be able to write honestly

- "Built an internal LLM gateway on EKS used by *N* teams. Made 100% of LLM spend visible per team and absorbed a provider outage with zero failed user requests."
- "Served open-weight LLMs on EKS GPU spot capacity with Karpenter and KEDA, reaching $0.*xx* per 1M tokens at *N* tok/s, *X*% cheaper than our hosted API at our traffic level."
- "Shipped a RAG runbook assistant with automated evals in CI and a documented threat model."

06 · EXECUTION

Build in public

Visibility compounds. A post with real numbers gets found by recruiters, tool maintainers and hiring managers long after you publish it. Post every one or two weeks, and every post needs one measured result.

#	WEEK	POST	HOOK (THE NUMBER)
1	W4	How much GPU does a 7B model need?	A VRAM table you computed and checked
2	W12	We put every LLM call behind one gateway	Spend per team, before and after
3	W18	Self-hosting an LLM on EKS spot GPUs: real costs	$ per 1M tokens vs. the API, with the break-even point
4	W22	Evals in CI for a RAG assistant	A score that caught a real regression
5	W24	Where our GPU money was wasted	Idle GPU hours found and removed

POST STRUCTURE

1. The problem, in one sentence
2. The setup (architecture diagram)
3. The numbers (a table or chart)
4. What surprised you
5. Repo link and what you'd do next

WHERE TO POST

- LinkedIn: a short version with the diagram
- Blog (Hashnode or dev.to): the full version
- GitHub: a README with reproduction steps
- The tool's community (vLLM, LiteLLM, CNCF Slack) when you find a bug or an insight

07 · EXECUTION

Weekly routine (about 9 hours)

DAY	TIME	ACTIVITY
Mon, Wed, Fri	3 × 1.5 h	Learn: docs, a course chapter or a paper, and take notes in your repo

DAY	TIME	ACTIVITY
Saturday	3 h	Build: one focused hands-on session on cloud GPUs or EKS, then `terraform destroy`
Sunday	1.5 h	Write: update the README, draft the post, plan next week

PITFALLS TO AVOID

- **Tutorial hell.** Watching without building. Follow the rule of one hour learning for every hour building.
- **Chasing every new tool.** The stack in section 03 is enough. Read about new tools, but don't switch plans for them.
- **Skipping the write-up.** An unpublished project is invisible to anyone hiring.
- **Forgetting a GPU running.** A forgotten g5 node costs about $24 a day. See the next section.

08 · EXECUTION

Budget and cost control

Prices are approximate, in US dollars. Check current AWS and RunPod prices before each phase.

ITEM	WHEN	APPROX. PER MONTH
LLM API tokens (OpenAI, Bedrock)	P1, P3, P5	$3–6
Spot VMs for kubeadm practice	P2	$4–7
EKS control plane ($0.10/h, only while practicing)	P3–P6	$5–10
Spot GPU hours (g5/g6 or RunPod, ~$0.40–1.00/h)	P4–P6	$18–35
Typical total		**$25–60**
CKA exam, one time (often 30–50% off in sales)	W8	~$445 once

Set up a budget alarm before you start any GPU work. This is also your first piece of FinOps-as-code to show in interviews.

HCL budget.tf

```hcl
resource "aws_budgets_budget" "learning" {
  name         = "ai-learning-monthly"
  budget_type  = "COST"
  limit_amount = "50"
  limit_unit   = "USD"
  time_unit    = "MONTHLY"

  notification {
    comparison_operator        = "GREATER_THAN"
    threshold                  = 80
    threshold_type             = "PERCENTAGE"
    notification_type          = "ACTUAL"
    subscriber_email_addresses = ["you@example.com"]
  }
}
```

09 · EXECUTION

Certifications

Get one or two certifications, chosen to close a gap. Projects carry more weight with hiring managers, but certifications help you pass résumé filters.

REQUIRED · WEEK 8

CKA, Certified Kubernetes Administrator

Closes your biggest gap. It's hands-on and widely recognized for platform roles.

OPTIONAL · AFTER P4

AWS Certified Machine Learning Engineer – Associate

Useful if you target AWS-heavy companies or roles that touch SageMaker.

OPTIONAL

NVIDIA AI Infrastructure and Operations (associate)

A signal for GPU-focused infrastructure roles. Take it only if job postings you want mention it.

10 · REFERENCE

Interview preparation

AI infrastructure interviews combine Kubernetes depth with system design for inference. Practice answering each of these out loud, drawing the architecture as you go.

QUESTION	WHAT A STRONG ANSWER COVERS
Design an LLM platform for 50 internal teams.	Gateway, keys and budgets per team, routing between hosted and self-hosted models, observability, cost showback, data privacy
A vLLM pod takes 6 minutes to become ready. Fix it.	Image size, weight download vs. local cache, pre-pulled images, a warm pool, readiness vs. startup probes
GPU utilization is 15% but the bill is high.	DCGM metrics, batching, right-sizing, MIG and time-slicing, consolidation, scale to zero, spot instances
Should we self-host or use an API?	Traffic volume, cost-per-token break-even, latency, data residency, engineering effort, model quality
A spot GPU gets interrupted mid-request.	The two-minute notice, draining, retries at the gateway, an on-demand fallback pool, PodDisruptionBudgets
How do you roll out a new model version safely?	Eval gates, shadow traffic, canary by weight at the gateway, rollback, GitOps

11 · REFERENCE

Top 1% checklist

By week 26, check each of these honestly. Every item is something most candidates for these roles can't claim.

- [ ] I run an AI system in production at my company, and people there know me for it.
- [ ] I can calculate VRAM needs and cost per 1M tokens on a whiteboard.
- [ ] I have served an open model on Kubernetes GPUs with autoscaling, and I measured cold start.
- [ ] I have published benchmarks with my own numbers and a reproducible repo.
- [ ] I can explain when self-hosting is cheaper than an API, and when it isn't.
- [ ] My RAG project has evals in CI that have caught at least one regression.
- [ ] I have a written threat model for an LLM application.
- [ ] I hold the CKA.

- [ ] I have published 5 or more posts, each with a measured result.
- [ ] I have contributed at least one issue, docs fix or pull request to vLLM, LiteLLM, KEDA, Karpenter or a similar project.

12 · REFERENCE

Resources

PHASE	RESOURCE	USE IT FOR
P1	Automate the Boring Stuff with Python (https://automatetheboringstuff.com)	Python basics, free online
P1	FastAPI tutorial (https://fastapi.tiangolo.com/tutorial/)	Building APIs
P1	Hugging Face LLM course (https://huggingface.co/learn/llm-course)	Tokens, transformers, models
P1	*AI Engineering* by Chip Huyen (O'Reilly)	The best single book on building with LLMs
P2	Kubernetes documentation (https://kubernetes.io/docs/home/) and Killercoda (https://killercoda.com)	CKA study and free practice labs
P3	LiteLLM docs (https://docs.litellm.ai)	Gateway, proxy config, virtual keys
P3	Langfuse docs (https://langfuse.com/docs)	LLM tracing and cost tracking
P4	vLLM docs (https://docs.vllm.ai)	Serving, metrics, benchmarks
P4	AI on EKS (AWS Labs) (https://awslabs.github.io/ai-on-eks/)	Reference Terraform for GPU EKS
P4	Karpenter docs (https://karpenter.sh/docs/) and KEDA docs (https://keda.sh/docs/)	GPU node pools and autoscaling
P4	NVIDIA GPU Operator (https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/index.html)	Drivers, device plugin, DCGM
P5	OWASP Top 10 for LLM Applications (https://genai.owasp.org/llm-top-10/)	Threat modelling
P6	KServe (https://kserve.github.io/website/)	The next serving layer to learn